

Computer and Network Security

Lab 01 (“Shallow Inspection”)

Disclaimer. Hacking without permission is illegal. These labs are strictly educational for learning about cyber-security in the areas of ethical hacking and penetration testing so that we can protect ourselves against the real hackers. Hence, any attack technique, tools, and technology, learned during these labs should only be launched on systems you either own or have permission to launch attacks on. Don’t wrong your classmates by launching attacks against them—they will know in the judgement day and you will pay for that. Please be professional, wise, and mature.

Lab Note. During the first three lectures, you have discussed the concept of security vulnerabilities and you have learned that a vulnerability comes either as a security flaw or a weakness in the system. We have discussed that most communication protocols that we use everyday have weaknesses as vulnerabilities. These weaknesses can be exploited to abuse the protocols and break security principles (e.g., disclosing secret and private information). In this lab, we will try to demonstrate that by attacking some communication protocols. Apart from learning hacking techniques, the goal of this lab is to sensitize you and make you aware of the danger through protocols inspection.

Description. In this lab, you will perform eavesdropping attacks. You will use Wireshark as a software tool to sniff packets related to certain protocols and analyze their content. You will exploit the fact that certain application-level protocols lack encryption to disclose sensitive information, e.g., credentials. We will consider the following protocols: HTTP, Telnet, SSH, and FTP. To that end, make sure to have:

- Two computers (or two virtual machines) — a server and a client.
- Wireshark installed, up and running (on the server).
- Web live server up and running (e.g., VS code IDE live server).
- Telnet, ftp, and ssh installed on both computers (telnet client and server, ssh client and server, and ftp client and server).

Task 1 (HTTP — Web Server)

[You can do this part on the localhost] Create a simple website composed of five webpages (e.g., A, B, C, D, and E). Page A (being index.html) should contain two hyperlinks pointing to page B and C, respectively. Pages B and C contain a form to fill up with **username** and **password**. Page B uses HTTP GET method to transfer the inputs to page D, whereas page C uses HTTP POST method to transfer the input to page E. Pages D and E just display “Welcome **username**”. Use a live server to serve the users of the local network with this website.

1. Start Wireshark and the web live-server, then browse the index page to fill out the forms on Page B and Page C, and submit the credentials (**username** and **password**)— using GET on Page B and POST on Page C. Note that if you use VS Code live server, the POST request will be sent to the loopback but will not be interpreted by the server (you will get an HTTP ERROR 405). However, this will not prevent capturing the POST request on Wireshark.
2. On Wireshark, decapsulate the HTTP header and look for the **Request URI** field, and the disclose the content of the fields **Request URI parameters**.
3. On Wireshark, decapsulate the HTTP header and look for the **HTML Form URL Encoded** field, and the disclose the content of the fields **Form items**.
4. What do you conclude? Would you continue using HTTP with GET and POST methods in their standard form?
5. Secure the website to use HTTPS (HTTP over TLS— *I know we have not done TLS yet, but I am just giving you a taste of what you are using everyday*), then repeat the above. What do you conclude? You need to create an RSA private key, its corresponding public key — x508 certificate, and configure the live server to use HTTP over TLS. Follow the steps in ANNEX 1.

Task 2 (Telnet — Remote Access to a Server)

Telnet is an application-level protocol in the TCP/IP suite. It allows a computer (client) to remotely connect to a server (running telnet on port 23) and execute commands on server. To that end, the user has to login using username and password. The password is sent in multiple TCP segments (each character in a TCP segment).

1. On the server computer, run Wireshark to capture packets flowing from and to the telnet server. You may use a Wireshark filter to only display related packets. Also, on the client's computer, use a telnet client to connect to the server.
2. On Wireshark you should see the exchanged Telnet messages. Analyze the messages and try to find out the username and password introduced by the user when they were logging in. What do you conclude? Do you know an alternative protocol for remote secure access?

Task 3 (SSH — Remote Secure Access to a Server)

SSH (Secure Shell) is a network protocol used for secure remote access to systems and secure file transfers between systems. It allows a computer (ssh client) to remotely connected through secure channel to a server (running ssh on port 22) in a client-server architecture, allowing users to access and manage remote systems securely.

1. On the server computer, run Wireshark to capture packets flowing from and to the ssh server (you can run it on the client too). You may use a Wireshark filter to only display related packets. Also on the client's computer, use an ssh client to connect to the server.
2. On Wireshark you should see the exchanged messages. Analyze the messages and try to find out the username and password introduced by the user when they were logging in. What do you conclude?

Task 4 (FTP — File transfer from and to an FTP Server)

FTP is a file transfer protocol. A client uses an ftp-client tool (e.g., through command line, or FileZilla) to connect and transfer (upload or download) a file from a remote server running an ftp service. The user can download a file using the **get** command and upload a file using **put** command. Of course, you must first login to the server (check ANNEX 3 for help).

1. On the server computer, run Wireshark to capture packets flowing from and to the ftp server (**vsftpd** in Linux). You may use a Wireshark filter to only display related packets.
2. On the client's computer, use an ftp client (e.g., **ftp 192.168.1.5**), to connect to the ftp server. Then, make a file transfer (upload or download).

3. On Wireshark you should see the exchanged ftp messages. Analyze the messages and try to find out the username and password introduced by the user when they were logging in. Could you read the content of the file being transferred? Could you try with a image? What do you conclude? What is the alternative for a secure file transfer?
4. You could set up the ftp configure file to use SSL protocol. You would need to create a private key and a certificate. I leave this question for you to do at home.

ANNEX 1: Set up VS Code LiveServer to use HTTPS

To use TSL over HTTP, do the following (make sure to have OpenSSL installed though— It is the toolset that allows you to create cryptographic keys):

- Within the folder of your website, create a new one— name it **keys**.
- Move to the new folder and create a 2048-bit RSA private key by typing:

```
openssl genrsa -aes128 -out key.key 2048
```

- Next, we create the RSA public key (for the private key that you previously created) and place that in a digital certificate. You will be prompted to input some information that will be written in the certificate. Make sure to put the value **localhost** when prompted for **Common Name** parameter. Also, the country code must only have two characters (e.g., DZ):

```
openssl req -days 18250 -new -newkey rsa:2048 -key key.key  
-x509 -out cert.pem
```

- At this point, you will noticed that two files have been created, one with extension **.key** (private key) and one with extension **.pem** (digital x508 certificate).
- In the directory of your project create a **.vscode** folder (dont' forget the dot— this is a hidden folder), and inside the folder create a **settings.json** file. Open the file using a text editor (e.g., **nano**) and write down the following code:

```
{  
  "liveServer.settings.root":  "/",  
  "liveServer.settings.https": {  
    "enable":  true,
```

```
"cert": "path to your cert.pem file",  
"key": "path to your key.key file",  
"passphrase": "your password of the private key"  
} }
```

- Done! close the server (VS Code) and re-open everything then run the server. It should use HTTPS (HTTP over TLSv1.3) now.

ANNEX 2: Install and Configure SSH server on GNU/Linux

OpenSSH (Open Secure Shell) is an open-source implementation of the SSH (Secure Shell) protocol. It provides encrypted communication over a network, allowing secure access to remote systems. OpenSSH incorporates several tools, such as **sshd** (ssh server daemon), **ssh** (ssh client), **scp** (secure copy), **sftp** (FTP over SSH), and ssh key management components.

1. Install ssh server: `$sudo apt install openssh-server`.
2. Install ssh client: `$sudo apt install openssh-client`
3. You may want to change the servers configuration in `/etc/ssh/sshd_config`
4. Start the ssh server: `sudo sshd -t -f /etc/ssh/sshd_config`

ANNEX 3: Install and Configure FTP server on GNU/Linux

1. Update your package list to ensure you have the latest information about available packages. Run: `$sudo apt update`
2. Install the vsftpd (Very Secure FTP Daemon) ftp server: `$sudo apt install vsftpd`
3. The server starts automatically. You can check that: `$sudo systemctl status vsftpd`.
4. You must then modify the configuration file for vsftpd. Run `$nano /etc/vsftpd`.
5. Update the following lines:

- Uncomment **anonymous_enable** to allow anonymous users to connect to the server.
- To enable local users to access the FTP server, uncomment **local_enable**
- To allow local users upload file to the FTP server, uncomment **write_enable**.

6. Restart the server: **sudo systemctl restart vsftpd**.

7. We must then permit FTP traffic through the system's firewall. Use **iptables** command:

```
$iptables -A INPUT -p tcp --dport 21 -j ACCEPT
$iptables -A OUTPUT -p tcp --sport 20 -j ACCEPT
```

Make sure that communication is accepted on both ports now.

```
$iptables -L -n
```

8. To allow users to connect to your FTP server, you need to create FTP user accounts:

```
$sudo adduser student
```

9. The default directory to download files from and upload file to, is the **/srv/ftp**. Make sure to create a txt file “midterm.txt” to download it later from the client. Once connected, you will have to change directory to the latter.

10. To upload a file “a.txt” to the server, run: **ftp> put a.txt**. Make sure that you have write access to the folder **/srv/ftp/**. Also, to download a file “a.txt” from the server, run: **ftp> get a.txt**.